



Sistemas de Computación

Trabajo Practico 3.

Modo Protegido.

Grupo: NoTengoGrupo

Alumnos:

- Bernardi Mateo
- Regnicoli Gustavo
- Schreiner Federico

Docentes:

- Miguel Angel Solinas
- Javier Alejandro Jorge

Año: 2026

Consigna.

Los procesadores x86 mantienen compatibilidad con sus antecesores y para agregar nuevas funcionalidades deben ir “evolucionando” en el tiempo durante el proceso de arranque. Todos los CPUs x86 comienzan en modo real en el momento de carga (boot time) para asegurar compatibilidad hacia atrás, en cuanto se los energiza se comportan de manera muy primitiva, luego mediante comandos se los hace evolucionar hasta poder obtener la máxima cantidad de prestaciones posibles.

El modo protegido es un modo operacional de los CPUs compatibles x86 de la serie 80286 y posteriores. Este modo es el primer salto evolutivo de los x86. El modo protegido tiene un número de nuevas características diseñadas para mejorar la multitarea y la estabilidad del sistema, tales como la protección de memoria, y soporte de hardware para memoria virtual como también la conmutación de tareas.

Se divide en 4 partes

- Ejecucion de programas x86
- UEFI y coreboot
- Linker
- Modo Protegido.

Donde hay que responder las preguntas dadas en las presentaciones y desarrollar las actividades propuestas.

Antes de empezar

- Clonen este repositorio e inicializan los submódulos (ver README).

<https://gitlab.com/sistemas-de-computacion-2021/protected-mode-sdc>

- Y un repositorio de código muy interesante para el práctico que ya está incluido en el repositorio clonado.

<https://github.com/cirosantilli/x86-bare-metal-examples>

PARTE EJECUCIÓN DE PROGRAMAS X86

El objetivo es construir manualmente una imagen de arranque que pueda ser reconocida por el BIOS. Hacemos una lista de los pasos a seguir.

1. Creación de la imagen booteable.

Abrimos el terminal y colocamos.

```
printf '\364%509s\125\252' > main.img
```

Eso hace lo siguiente.

\364: Es el código octal para la instrucción HLT (Halt) en x86. El procesador se detendrá apenas arranque.

%509s: Rellena con 509 espacios en blanco (ceros/nulos) para completar el tamaño del sector.

\125\252: Es la firma mágica en octal (0x55 y 0xAA). Sin esto, el BIOS dirá "No bootable device found".

De esa forma generamos el MBR, (Master Boot Record.)

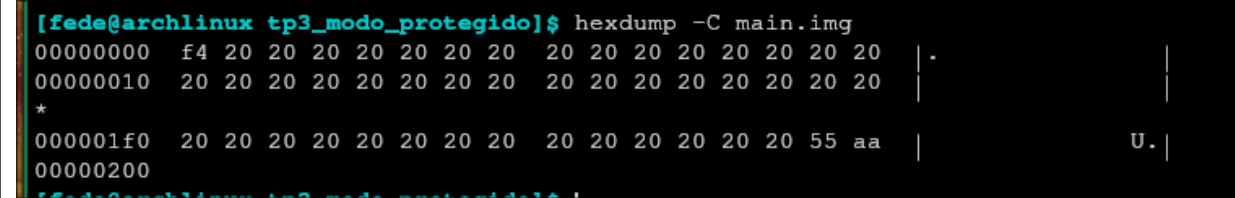
2. Compilamos el archivo. Y confirmamos.

Lo podemos hacer con hd (hay que instalarlo si no lo tenes.)

```
hd main.img
```

Pero esto no me funciona en la distro, entonces usamos hexdump

```
hexdump -C main.img
```



```
[fedora@archlinux tp3_mod0_protegido]$ hexdump -C main.img
00000000  f4 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 |. |
00000010  20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 |. |
*
000001f0  20 20 20 20 20 20 20 20 20 20 20 20 20 55 aa | U. |
00000200
```

Con esto, se verifica la estructura de main.img, para confirmar que respeta el standar MBR (Master Boot Record).

El archivo inicia en el byte f4 (que es la instruccion HLT). y luego se concluye con 55 aa.

Tambien podemos utilizar objdump

```
objdump -D -m i386 -M intel -b binary main.img
```

```
[fed@archlinux tp3_mod0_protegido]$ objdump -D -m i386 -M intel -b binary main.img

main.img:      file format binary


Disassembly of section .data:

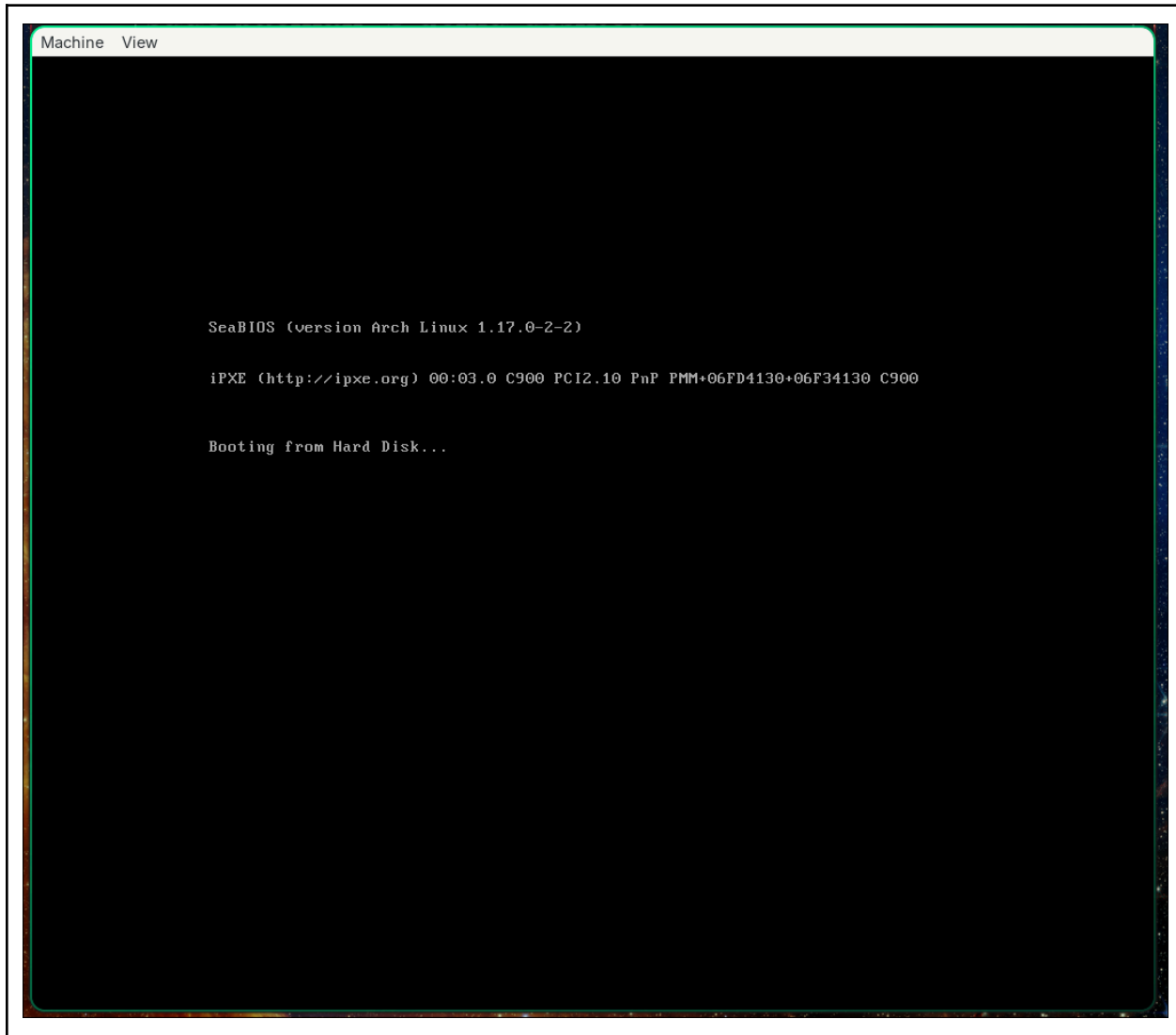
00000000 <.data>:
 0:  f4          hlt
 1:  20 20      and     BYTE PTR [eax], ah
 3:  20 20      and     BYTE PTR [eax], ah
 5:  20 20      and     BYTE PTR [eax], ah
 7:  20 20      and     BYTE PTR [eax], ah
 9:  20 20      and     BYTE PTR [eax], ah
 b:  20 20      and     BYTE PTR [eax], ah
 d:  20 20      and     BYTE PTR [eax], ah
 f:  20 20      and     BYTE PTR [eax], ah
11:  20 20      and     BYTE PTR [eax], ah
13:  20 20      and     BYTE PTR [eax], ah
15:  20 20      and     BYTE PTR [eax], ah
17:  20 20      and     BYTE PTR [eax], ah
19:  20 20      and     BYTE PTR [eax], ah
1b:  20 20      and     BYTE PTR [eax], ah
1d:  20 20      and     BYTE PTR [eax], ah
1f:  20 20      and     BYTE PTR [eax], ah
21:  20 20      and     BYTE PTR [eax], ah
23:  20 20      and     BYTE PTR [eax], ah
25:  20 20      and     BYTE PTR [eax], ah
27:  20 20      and     BYTE PTR [eax], ah
29:  20 20      and     BYTE PTR [eax], ah
2b:  20 20      and     BYTE PTR [eax], ah
2d:  20 20      and     BYTE PTR [eax], ah
2f:  20 20      and     BYTE PTR [eax], ah
31:  20 20      and     BYTE PTR [eax], ah
33:  20 20      and     BYTE PTR [eax], ah
35:  20 20      and     BYTE PTR [eax], ah
37:  20 20      and     BYTE PTR [eax], ah
39:  20 20      and     BYTE PTR [eax], ah
3b:  20 20      and     BYTE PTR [eax], ah
```

Está herramienta objdump, podemos interpretar los bytes de la imagen como instrucciones ejecutables de arquitectura i386 utilizando la sintaxis de intel. Por ejemplo, vemos que el primer byte f4 es identificado como la instruccion hlt. Lo que valida la logica.

3. Testeamos con qemu (antes tengo que instalarlo)

```
sudo pacman -S qemu-desktop
```

```
qemu-system-i386 -drive format=raw,file=main.img
```



Donde se verifica de forma funcional mediante el emulador QEMU, ejecutando la imagen. El sistema se inicia y se detiene luego del mensaje "Booting from Hard Disk", lo que confirma que la bios reconoció el sector como booteable y la instrucción fue cargada y ejecutada correctamente.

PARTE UEFI Y COREBOOT.

La especificación UEFI (Unified Extensible Firmware Interface) surgió como la evolución necesaria para superar las limitaciones del antiguo BIOS. Actúa como una capa intermedia de software que gestiona la comunicación entre el firmware del hardware y el sistema operativo durante el arranque.

Una de sus grandes ventajas es que ofrece un entorno mucho más flexible. Por ejemplo, incluye el UEFI Interactive Shell, una consola que permite ejecutar scripts con extensión .efi. Además, permite a los desarrolladores programar aplicaciones en lenguaje C que corren directamente sobre el firmware, sin necesidad de un SO cargado.

En términos de programación, los desarrolladores pueden interactuar con la pantalla mediante llamadas a funciones específicas como SystemTable->ConOut->OutputString, facilitando el despliegue de información antes de que tome el control el núcleo del sistema operativo.

El firmware UEFI representa un objetivo crítico para los atacantes debido a que opera con los privilegios más altos del sistema y se ejecuta antes de cualquier solución de seguridad del sistema operativo. Hay diferentes ataques como pueden ser

- Ataques por saturación de memoria: Ocurren cuando un componente del firmware no valida el tamaño de los datos que recibe. Al enviar más información de la permitida, se logra sobrescribir la pila de ejecución con código malicioso para tomar el control total del procesador antes de que el sistema operativo inicie.
- Evasión de la cadena de confianza: Se basa en romper la verificación de firmas digitales, ya sea explotando fallas en la lógica de validación o forzando el uso de cargadores de arranque antiguos que, aunque son oficiales, tienen vulnerabilidades conocidas que permiten cargar un núcleo del sistema alterado.
- Reprogramación del chip físico: Aprovecha que algunos firmwares dejan los bits de protección contra escritura desprotegidos o fallan al bloquear los registros de configuración. Esto permite a un atacante reprogramar directamente el chip de la placa base con una versión maliciosa que persiste incluso si se cambia el disco duro.

El CSME es un subsistema de seguridad embebido directamente en el hardware de Intel que opera de forma autónoma y aislada del sistema operativo principal. Su función es encargarse de la verificación del arranque seguro y la gestión de claves criptográficas. Al ejecutarse en un procesador independiente, mantiene su integridad incluso si el kernel del sistema operativo ha sido comprometido por un atacante.

Por otro lado, el Intel MEBx (Management Engine BIOS Extension) es la interfaz de configuración a nivel de firmware que permite interactuar con las funciones del motor de administración de Intel. A través de

este módulo, los administradores pueden habilitar tecnologías para el control remoto del equipo incluso cuando este se encuentra apagado. Es el panel crítico donde se gestionan las credenciales y los parámetros de red que definen cómo se comportará el subsistema de gestión a bajo nivel.

El Coreboot es un firmware de código abierto diseñado para inicializar el hardware de forma minimalista, delegando la carga del sistema operativo a un componente secundario llamado payload. Al eliminar procesos superfluos del BIOS tradicional, ofrece una transparencia total que permite auditar el código en busca de vulnerabilidades y optimizar drásticamente la velocidad de arranque. Su naturaleza modular lo convierte en el estándar para dispositivos orientados a la privacidad y seguridad, como las laptops de System76 o las Chromebooks.

PARTE LINKER

El Linker es una herramienta fundamental del toolchain de compilación que integra múltiples archivos objeto en un único archivo ejecutable o imagen binaria. Su función principal es resolver las referencias cruzadas entre módulos y asignar direcciones de memoria absolutas a los símbolos, garantizando que el hardware localice cada instrucción con precisión. En entornos bare-metal, el linker es el responsable de estructurar el binario final bajo las restricciones específicas de memoria requeridas para que el firmware o BIOS pueda cargarlo directamente.

Dado el siguiente script del linker

```
SECTIONS
{
    /* The BIOS loads the code from the disk to this location.
     * We must tell that to the linker so that it can properly
     * calculate the addresses of symbols we might jump to.
     */
    . = 0x7c00;
    .text :
    {
        __start = .;
        *(.text)
        /* Place the magic boot bytes at the end of the first 512 sector. */
        . = 0x1FE;
        SHORT(0xAA55)
    }
}
```

La dirección 0x7C00 especificada en el script del linker define el punto de carga base en la memoria RAM donde el BIOS deposita el primer sector del disco. Es fundamental declarar este valor para que el enlazador calcule correctamente las direcciones absolutas de los saltos (jmp) y la ubicación de las variables. Sin esta referencia, cualquier acceso a símbolos del programa fallaría, ya que el código se ejecutaría en una posición de memoria distinta a la prevista durante la compilación.

Desarrollamos una comparación entre objdump con hd,

hd (o hexdump): Muestra los bytes "crudos" o planos del archivo en hexadecimal y ASCII. Permite confirmar que la firma de booteo 55 aa esté al final (offset 0x1FE) y que el código empiece en el offset 0 del archivo.

objdump: Interpreta esos mismos bytes y los "traduce" a lenguaje ensamblador humano. Si se usa el flag --adjust-vma=0x7C00, permite verificar que las direcciones de memoria coincidan con lo que el CPU verá durante la ejecución real.

Y de forma practica.

Usando hex dump.

Hexdump main.img

```
[fede@archlinux 01HelloWorld]$ hexdump main.img
00000000 0fbec b47c ac0e c008 0474 10cd f7eb 68f4
00000010 6c65 6f6c 7720 726f 646c 6600 0f2e 841f
00000020 0000 0000 6600 0f2e 841f 0000 0000 6600
00000030 0f2e 841f 0000 0000 6600 0f2e 841f 0000
00000040 0000 6600 0f2e 841f 0000 0000 6600 0f2e
00000050 841f 0000 0000 6600 0f2e 841f 0000 0000
00000060 6600 0f2e 841f 0000 0000 6600 0f2e 841f
00000070 0000 0000 6600 0f2e 841f 0000 0000 6600
00000080 0f2e 841f 0000 0000 6600 0f2e 841f 0000
00000090 0000 6600 0f2e 841f 0000 0000 6600 0f2e
000000a0 841f 0000 0000 6600 0f2e 841f 0000 0000
000000b0 6600 0f2e 841f 0000 0000 6600 0f2e 841f
000000c0 0000 0000 6600 0f2e 841f 0000 0000 6600
000000d0 0f2e 841f 0000 0000 6600 0f2e 841f 0000
000000e0 0000 6600 0f2e 841f 0000 0000 6600 0f2e
000000f0 841f 0000 0000 6600 0f2e 841f 0000 0000
00000100 6600 0f2e 841f 0000 0000 6600 0f2e 841f
00000110 0000 0000 6600 0f2e 841f 0000 0000 6600
00000120 0f2e 841f 0000 0000 6600 0f2e 841f 0000
00000130 0000 6600 0f2e 841f 0000 0000 6600 0f2e
00000140 841f 0000 0000 6600 0f2e 841f 0000 0000
00000150 6600 0f2e 841f 0000 0000 6600 0f2e 841f
00000160 0000 0000 6600 0f2e 841f 0000 0000 6600
00000170 0f2e 841f 0000 0000 6600 0f2e 841f 0000
00000180 0000 6600 0f2e 841f 0000 0000 6600 0f2e
00000190 841f 0000 0000 6600 0f2e 841f 0000 0000
000001a0 6600 0f2e 841f 0000 0000 6600 0f2e 841f
000001b0 0000 0000 6600 0f2e 841f 0000 0000 6600
000001c0 0f2e 841f 0000 0000 6600 0f2e 841f 0000
000001d0 0000 6600 0f2e 841f 0000 0000 6600 0f2e
000001e0 841f 0000 0000 6600 0f2e 841f 0000 0000
000001f0 6600 0f2e 841f 0000 0000 0f00 001f aa55
00002000 0004 0000 0020 0000 0005 0000 4e47 0055
00002010 0002 c001 0004 0000 0001 0000 0000 0000
00002020 0001 c001 0004 0000 0001 0000 0000 0000
00002030
```


Usando objdump

```
objdump -D -b binary -m i386 --adjust-vma=0x7C00 main.img
```

```
[fede@archlinux 01HelloWorld]$ objdump -D -b binary -m i386 --adjust-vma=0x7C00 main.img
```

```
main.img:      file format binary
```

```
Disassembly of section .data:
```

```
00007c00 <.data>:
```

7c00:	be 0f 7c b4 0e	mov	\$0xeb47c0f,%esi
7c05:	ac	lods	%ds:(%esi),%al
7c06:	08 c0	or	%al,%al
7c08:	74 04	je	0x7c0e
7c0a:	cd 10	int	\$0x10
7c0c:	eb f7	jmp	0x7c05
7c0e:	f4	hlt	
7c0f:	68 65 6c 6c 6f	push	\$0x6f6c6c65
7c14:	20 77 6f	and	%dh,0x6f(%edi)
7c17:	72 6c	jb	0x7c85
7c19:	64 00 66 2e	add	%ah,%fs:0x2e(%esi)
7c1d:	0f 1f 84 00 00 00 00	nopl	0x0(%eax,%eax,1)
7c24:	00		
7c25:	66 2e 0f 1f 84 00 00	nopw	%cs:0x0(%eax,%eax,1)
7c2c:	00 00 00		
7c2f:	66 2e 0f 1f 84 00 00	nopw	%cs:0x0(%eax,%eax,1)
7c36:	00 00 00		
7c39:	66 2e 0f 1f 84 00 00	nopw	%cs:0x0(%eax,%eax,1)
7c40:	00 00 00		
7c43:	66 2e 0f 1f 84 00 00	nopw	%cs:0x0(%eax,%eax,1)
7c4a:	00 00 00		
7c4d:	66 2e 0f 1f 84 00 00	nopw	%cs:0x0(%eax,%eax,1)
7c54:	00 00 00		
7c57:	66 2e 0f 1f 84 00 00	nopw	%cs:0x0(%eax,%eax,1)
7c5e:	00 00 00		
7c61:	66 2e 0f 1f 84 00 00	nopw	%cs:0x0(%eax,%eax,1)
7c68:	00 00 00		
7c6b:	66 2e 0f 1f 84 00 00	nopw	%cs:0x0(%eax,%eax,1)
7c72:	00 00 00		
7c75:	66 2e 0f 1f 84 00 00	nopw	%cs:0x0(%eax,%eax,1)
7c7c:	00 00 00		
7c7f:	66 2e 0f 1f 84 00 00	nopw	%cs:0x0(%eax,%eax,1)
7c86:	00 00 00		
7c89:	66 2e 0f 1f 84 00 00	nopw	%cs:0x0(%eax,%eax,1)
7c90:	00 00 00		
7c93:	66 2e 0f 1f 84 00 00	nopw	%cs:0x0(%eax,%eax,1)
7c9a:	00 00 00		
7c9d:	66 2e 0f 1f 84 00 00	nopw	%cs:0x0(%eax,%eax,1)
7ca4:	00 00 00		
7ca7:	66 2e 0f 1f 84 00 00	nopw	%cs:0x0(%eax,%eax,1)
7cae:	00 00 00		

```

7dda:    00 00 00
7ddd:    66 2e 0f 1f 84 00 00    nopw    %cs:0x0(%eax,%eax,1)
7de4:    00 00 00
7de7:    66 2e 0f 1f 84 00 00    nopw    %cs:0x0(%eax,%eax,1)
7dee:    00 00 00
7df1:    66 2e 0f 1f 84 00 00    nopw    %cs:0x0(%eax,%eax,1)
7df8:    00 00 00
7dfb:    0f 1f 00                nopl    (%eax)
7dfe:    55                      push    %ebp
7dff:    aa                      stos    %al,%es:(%edi)
7e00:    04 00                add     $0x0,%al
7e02:    00 00                add     %al,(%eax)
7e04:    20 00                and     %al,(%eax)
7e06:    00 00                add     %al,(%eax)
7e08:    05 00 00 00 47        add     $0x47000000,%eax
7e0d:    4e                      dec     %esi
7e0e:    55                      push    %ebp
7e0f:    00 02                add     %al,(%edx)
7e11:    00 01                add     %al,(%ecx)
7e13:    c0 04 00 00        rolb    $0x0,(%eax,%eax,1)
7e17:    00 01                add     %al,(%ecx)
7e19:    00 00                add     %al,(%eax)
7e1b:    00 00                add     %al,(%eax)
7e1d:    00 00                add     %al,(%eax)
7e1f:    00 01                add     %al,(%ecx)
7e21:    00 01                add     %al,(%ecx)
7e23:    c0 04 00 00        rolb    $0x0,(%eax,%eax,1)
7e27:    00 01                add     %al,(%ecx)
7e29:    00 00                add     %al,(%eax)
7e2b:    00 00                add     %al,(%eax)
7e2d:    00 00                add     %al,(%eax)
...
[fede@archlinux 01HelloWorld]$ |

```

La opción `--oformat binary` instruye al linker para generar un archivo binario plano (raw binary), eliminando encabezados, tablas de símbolos y metadatos propios de formatos complejos como ELF o PE.

En entornos de bajo nivel, esto es fundamental porque el BIOS carece de un cargador de archivos sofisticado y solo es capaz de ejecutar instrucciones de máquina puras de forma secuencial. Al omitir toda la estructura del sistema operativo, se garantiza que el primer byte del archivo corresponda exactamente a la primera instrucción que el procesador debe ejecutar en la dirección 0x7C00.

Ejecución de ejemplo helloworld con qemu

En esta sección vamos a realizar un procedimiento para compilar y ejecutar un programa básico “Hello World” en modo real de un procesador x86 (Se diseña;a para correr directamente sobre el hardware, Bare metal.). Vamos a generar una imagen binaria arrancable que es emulada mediante QEMU y depurada en tiempo real con GDB, permitiendo ver el flujo de ejecución desde la dirección inicio de la BIOS 0x7c00.

1. Clonamos el repo recomendado.

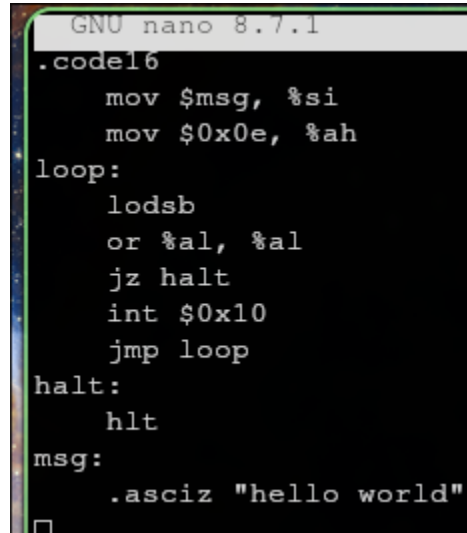
```
# Clona el repositorio oficial
git clone https://gitlab.com/sistemas-de-computacion-unc/protected-mode-sdc.git .

# Inicializa y descarga los ejemplos de Ciro Santilli (el submódulo)
git submodule init
git submodule update
```

2. Vamos a la carpeta de ejemplo.

```
cd 01HelloWorld
```

Donde el código del ejemplo es



```
GNU nano 8.7.1
.code16
    mov $msg, %si
    mov $0x0e, %ah
loop:
    lodsb
    or %al, %al
    jz halt
    int $0x10
    jmp loop
halt:
    hlt
msg:
    .asciz "hello world"
```

3. Ensamblamos

```
as -g -o main.o main.S
```

4. Enlazamos

```
ld --oformat binary -o main.img -T link.ld main.o
```

5. Ejecutamos.

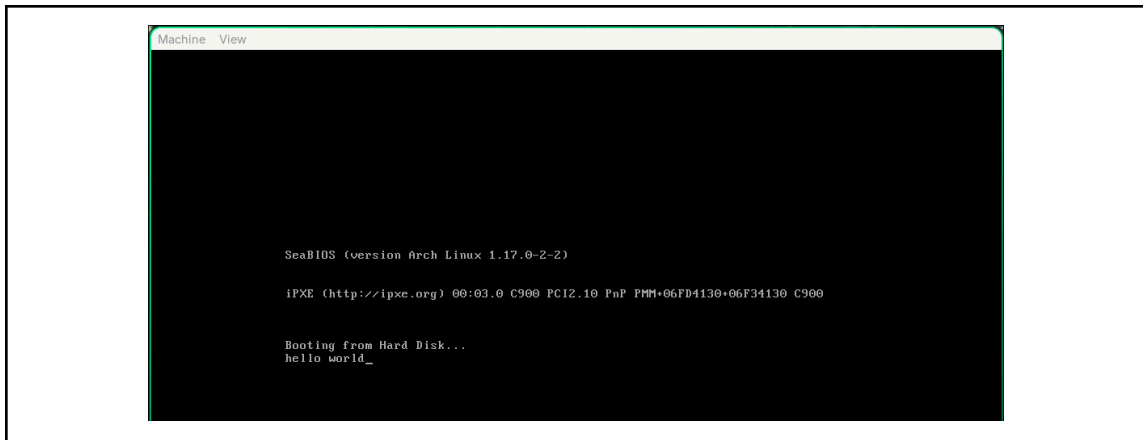
```
qemu-system-i386 -fda main.img -boot a -s -S -monitor stdio
```

Y al mismo tiempo, en otra terminal. Escribimos en cada línea separada y de forma individual

```
gdb gdb activate_protected_mode.elf  
  
(gdb) target remote localhost:1234  
  
(gdb) set architecture i8086  
  
(gdb) break *0x7c00  
  
(gdb) continue
```

Donde las capturas de lo realizado, son.

```
(gdb) set architecture i8086  
The target architecture is set to "i8086".  
(gdb) break *0x7c00  
Breakpoint 1 at 0x7c00  
(gdb) continue  
Continuing.  
  
Breakpoint 1, 0x00007c00 in ?? ()  
(gdb)
```



Lo cual confirma la correcta ejecución.

Test en Hardware Real.

Ahora, poniendo la imagen del primer ejemplo realizado en esta actividad en un pendrive, buscamos bootear desde este, configurando algunas opciones en la BIOS, para ver si corresponde con lo mismo dado en QEMU. Mostrando las correspondientes fotos.

1. Grabamos en el pen drive la imagen creada al principio del proyecto

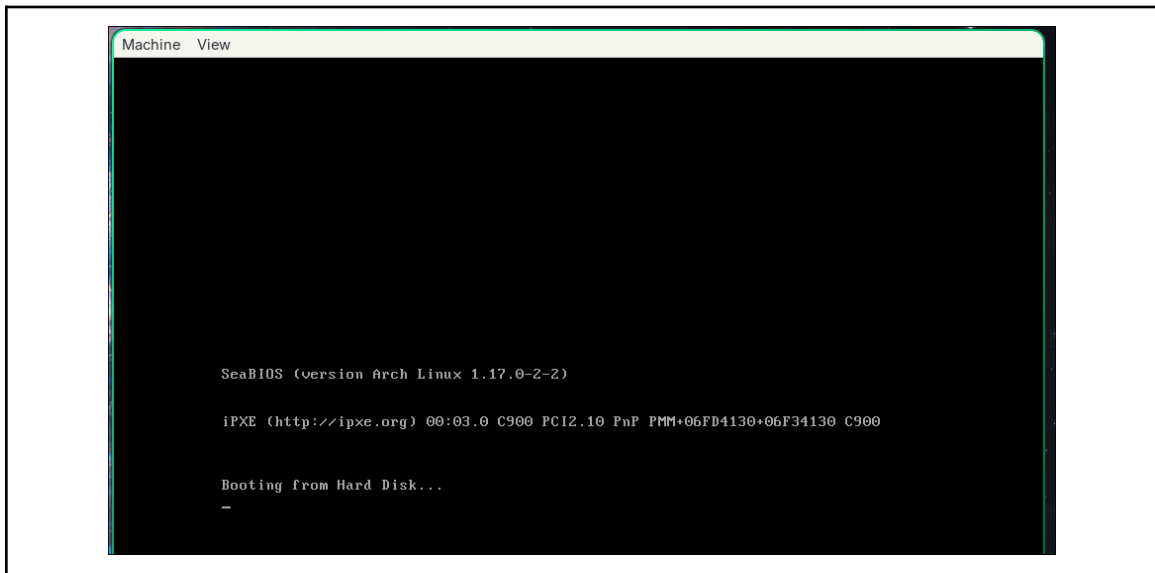
```
sudo dd if=hello.img of=/dev/sdd status=progress && sync
```

2. Verificación rápida sobre el pen drive "sdd"

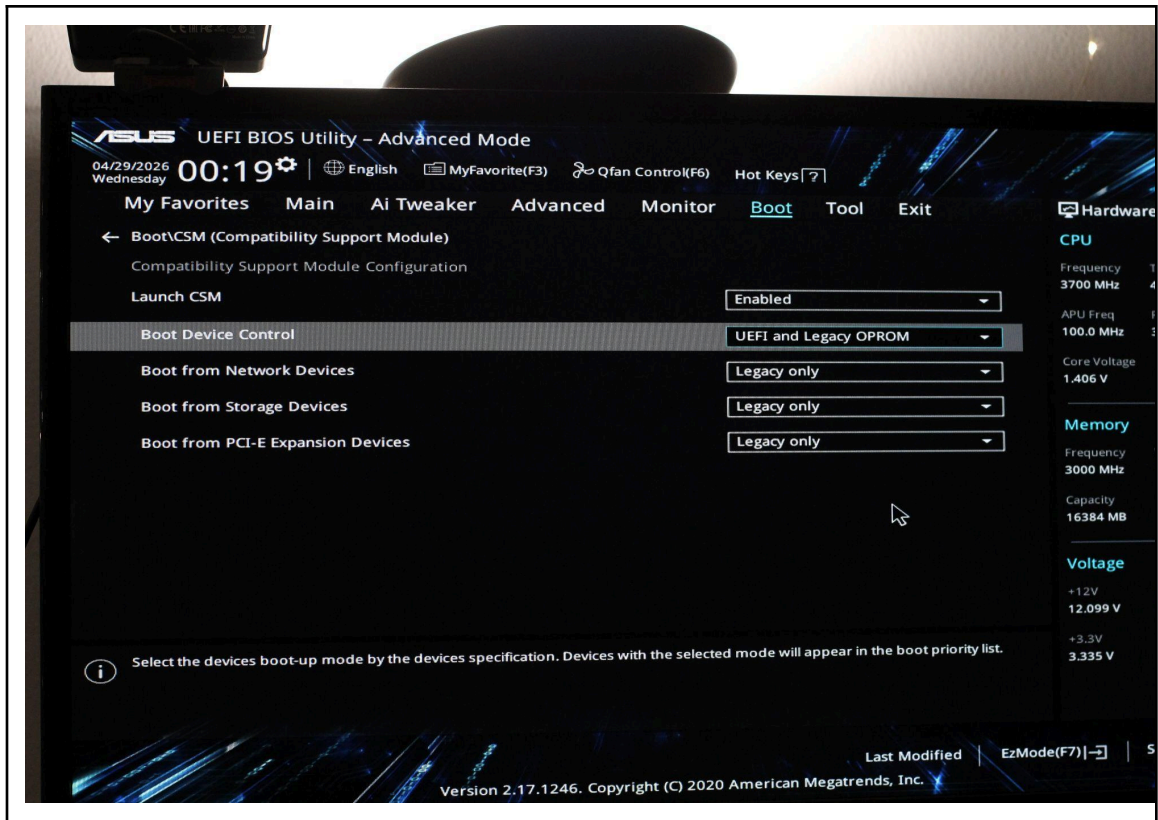
```
sudo hexdump -C -n 512 /dev/sdd
```

```
[fedex@archlinux tp3_mod0_protegido]$ sudo hexdump -C -n 512 /dev/sdd
00000000  f4 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 |.          |
00000010  20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 |          |
*
000001f0  20 20 20 20 20 20 20 20 20 20 20 20 20 55 aa      |          U.|
00000200
[fedex@archlinux tp3_mod0_protegido]$
```

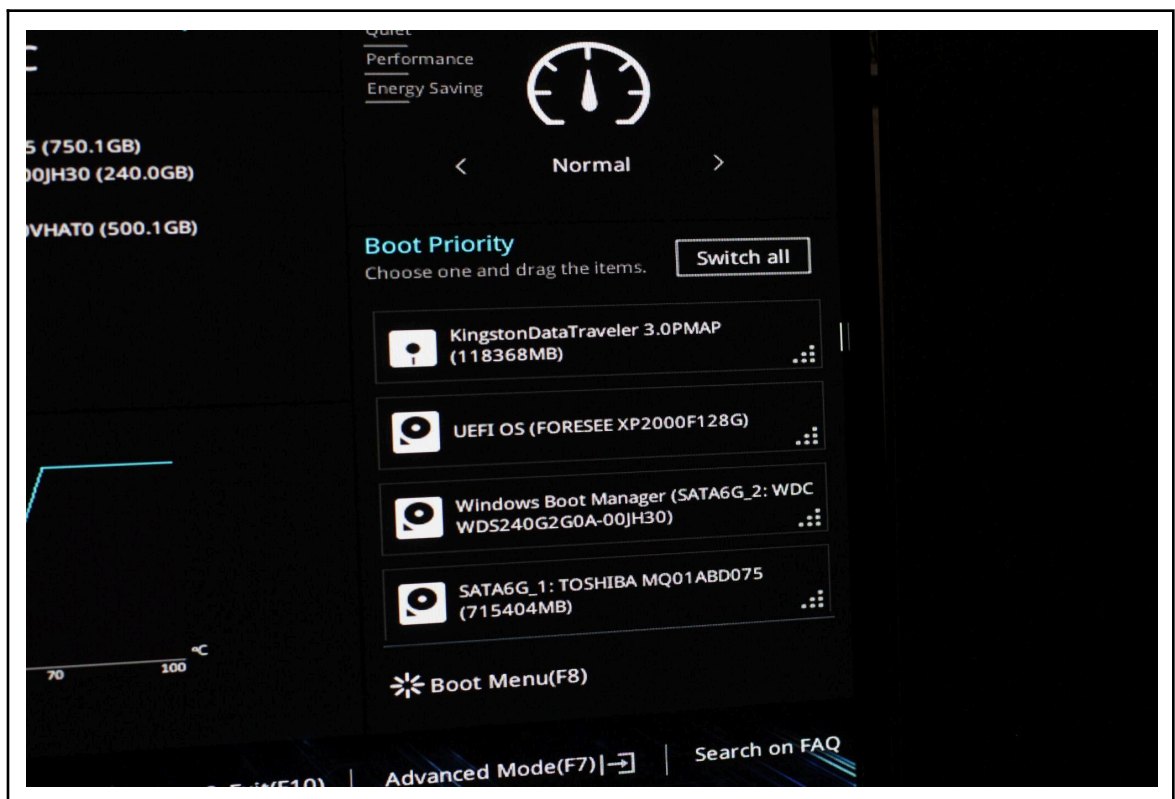
3. Verificamos con QEMU.



4. Ahora bien, el siguiente paso es desmontar el pen drive, entrar a la bios/boot menu. Configurar el legacy (modo UEFI este en Legacy o CSM) y bootear desde el pen drive,



Lo cual, configuramos que arranque desde el pendrive



Y arrancamos, dandonos lo mismo que con QEMU



PARTE DESAFIO FINAL.

En esta última etapa, se busca la transición de la arquitectura x86 al modo protegido. El objetivo es demostrar cómo el hardware, a través de descriptores de segmento diferenciados, impone restricciones de acceso a la memoria, y para ello, se necesita desarrollar un programa que active la protección del procesador provoque un conflicto de permisos, permitiendo observar la respuesta del sistema ante operaciones no autorizadas en segmentos de solo lectura.

1. Programa con descriptores diferenciados

En el modo real, la memoria es un espacio plano donde cualquier instrucción puede acceder a cualquier dirección. Al pasar al modo Protegido, introducimos una segmentación mediante descriptores. Un descriptor es una entrada en la GDT (global Descriptor Table) que le indica al hardware características como el límite, base y atributos de un bloque de memoria.

Descriptores Diferenciados: En lugar de que el código y los datos compartan las mismas reglas, definimos áreas con propósitos específicos:

- Descriptor de Código (CS): Se configura con permisos de ejecución y lectura. El hardware impedirá que el procesador intente escribir sobre sus propias instrucciones, evitando corrupciones accidentales.
- Descriptor de Datos (DS): Se configura con permisos de lectura y escritura. Al desplazar su Base (por ejemplo, a $\$0x00100000$), estamos creando un espacio de memoria físicamente separado del código.

Esto se utiliza para crear una jerarquía de protección.

2. Cambio a Solo Lectura y Verificación

Al configurar segmentos de datos, el parámetro crítico es el Byte de acceso. Al modificar este de 0x92 (lectura/escritura) a 0x90 (solo lectura), se desactiva el bit de escritura en el descriptor de la GDT. Cuando el procesador ejecuta la instrucción

```
movl $0xDEADBEEF, (0x0)
```

La unidad de segmentación consulta el descriptor indexado por el selector cargado en DS. Al detectar que se intenta una escritura en un segmento definido como solo lectura, el hardware bloquea la operación para preservar la integridad de la memoria.

Esta violación de privilegios dispara inmediatamente una excepción de Fallo de Protección General (GP). Debido a que el entorno actual no dispone de una IDT (Interrupt Descriptor Table) configurada, el procesador no encuentra un manejador para procesar el GP. Esta falla en cascada (al no poder manejar la excepción de la excepción) deriva en un Triple Fault, lo que fuerza al hardware a ejecutar un reinicio preventivo del sistema, fenómeno que se observa en la ejecución de QEMU más adelante.

3. Carga de Registros de Segmento

En la transición al modo protegido, la función de los registros de segmento (CS, DS, SS, etc.) cambia radicalmente para permitir el control de acceso y la seguridad de memoria.

Estos registros de segmentos se cargan con un selector de segmentos. Este valor no representa una dirección de memoria, sino un índice o puntero dentro de la GDT. El procesador utiliza este selector para buscar en la GDT la base, el límite y los derechos de acceso necesarios para calcular la dirección física y permitir que el hardware valide cada operación antes de que ocurra, haciendo que cualquier intento de escritura posterior será bloqueado por la unidad de segmentación antes de comprometer la integridad de la memoria.

4. Código Assembler (Sin Macros)

Tenemos recursos que podemos utilizar de ejemplos para crear este código.

<https://github.com/cirosantilli/x86-bare-metal-examples>

Necesitamos crear un programa en ensamblador que cambie al procesador al modo protegido, definiendo dos segmentos de memoria distintos (Código y datos) que están físicamente diferenciados. Y que además, restrinja los permisos de los segmentos de datos a “solo lectura” para observar el comportamiento al intentar escribir en lugares que esté protegido.

```
.code16
.global _start

_start:
    cli                # Desactivamos interrupciones
    lgdt gdt_descriptor

    # con esto pasamos a modo protegido
    mov %cr0, %eax
    or $1, %eax
    mov %eax, %cr0
    ljmp $0x08, $protected_mode

.code32
protected_mode:
    # Con esto cargamos los registros de segmento de datos con el selector 0x10
    mov $0x10, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %fs
    mov %ax, %gs
    mov %ax, %ss

    # intentamos escribir en segmento de solo lectura, provoca un GP
    movl $0xDEADBEEF, (0x0)

hang:
    jmp hang

# --- Estructura de la GDT ---
.align 4
gdt_start:
    .quad 0x0000000000000000    # Descriptor Nulo
```

```
# Selector 0x08: Código (Base 0, Límite 4GB, Ejec/Lect, Granularidad 4KB)
# Access byte: 0x9A (10011010b)
.word 0xffff, 0x0000, 0x9a00, 0x00cf

# Selector 0x10: Datos (Base 0, Límite 4GB, SOLO LECTURA, Granularidad 4KB)
# Access byte: 0x90 (10010000b) -> Bit de escritura en 0
.word 0xffff, 0x0000, 0x9000, 0x00cf
gdt_end:

gdt_descriptor:
.word gdt_end - gdt_start - 1
.long gdt_start
```

Y con este programa, configuramos y activamos el Modo Protegido del procesador, cargando una tabla, GDT, que define reglas estrictas sobre cómo se puede acceder a la memoria.

Luego, establecemos un segmento de datos específico con permisos de solo lectura, utilizando selectores en lugar de direcciones directas para organizar el espacio de trabajo en 32 bits.

Y por último, intentamos forzar una escritura en esa zona protegida para demostrar que el hardware detecta la infracción y bloquea la operación mediante un fallo de protección, GP.

Para testear el código.

1. En el terminal, vamos a la carpeta que vamos a desarrollar, abrimos el editor y guardamos el código

```
nano desafio.S
```

2. Ensamblamos el código objeto

```
as --32 desafio.S -o desafio.o
```

3. Enlazamos el código

```
ld -m elf_i386 -T link.ld -o desafio.elf desafio.o
```

4. Extraemos solo los datos binarios

```
objcopy -O binary desafio.elf desafio.bin
```

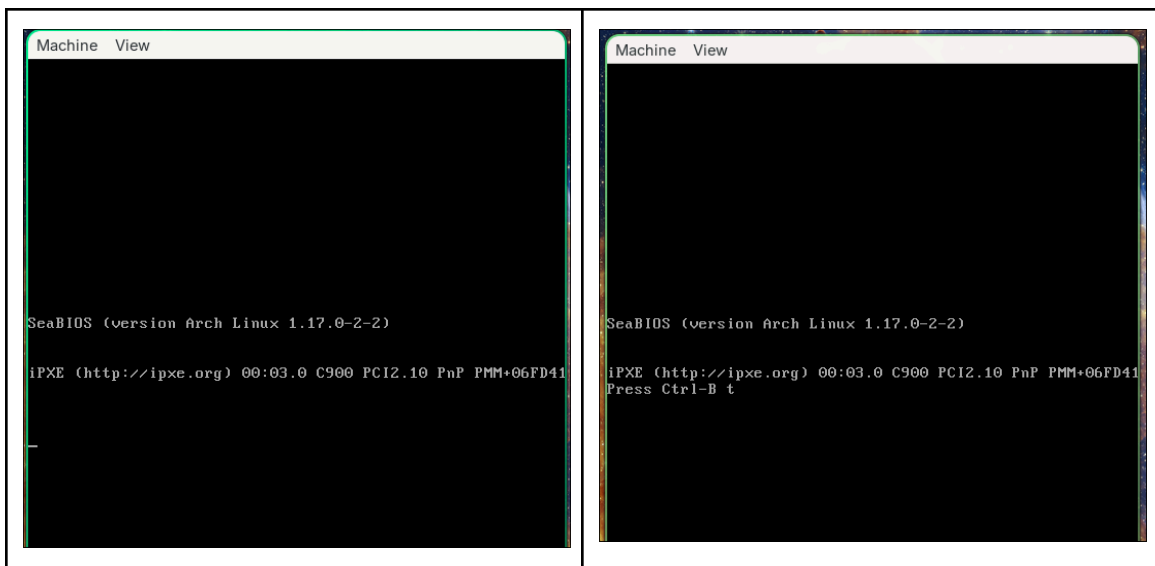
5. Crear la imagen final de 512 bytes con la firma de booteo

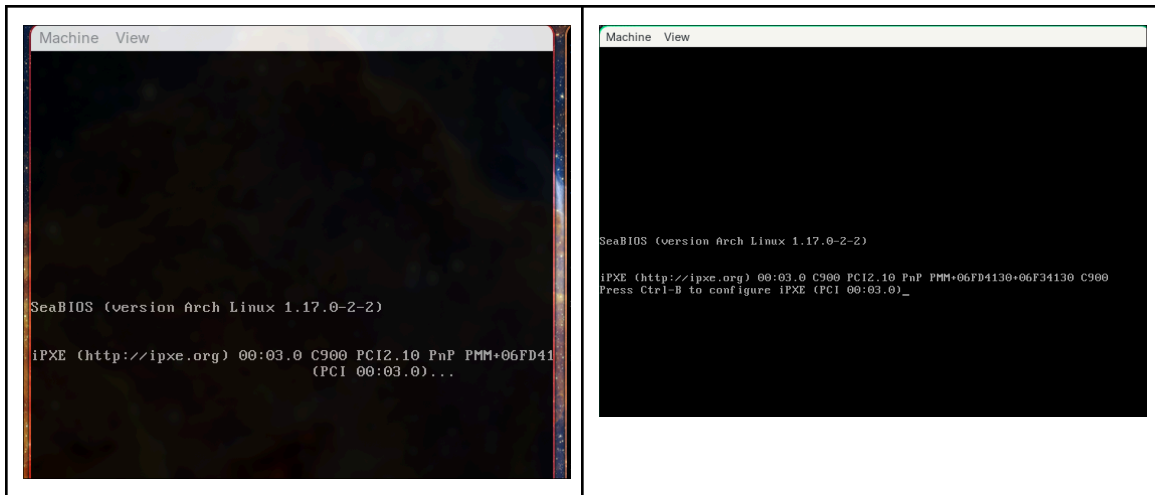
```
printf '\364%509s\125\252' > main.img  
dd if=desafio.bin of=main.img conv=notrunc
```

6. Verificación de "Fallo". Para comprobar que la protección de memoria está funcionando

```
qemu-system-i386 -drive format=raw,file=main.img -monitor stdio
```

Lo cual nos da las siguientes capturas.





La ventana de QEMU se abre y se reinicia constantemente. Lo cual quiere decir que el código entro en modo protegido, intento escribir en zona de solo lectura, el hardware detecto el error y reinicia la PC virtual porque no hay nadie que maneje ese error.

7. Verificación con GDB

Esto lo hacemos para ver los registros internos antes de que se produzca el conflicto de permisos de memoria que fuerce el reinicio del sistema. Para ello, en el terminal, ponemos

```
qemu-system-i386 -fda main.img -boot a -s -S
```

En otro terminal.

```
gdb desafio.elf
```

Y luego los siguientes comandos uno a la vez.

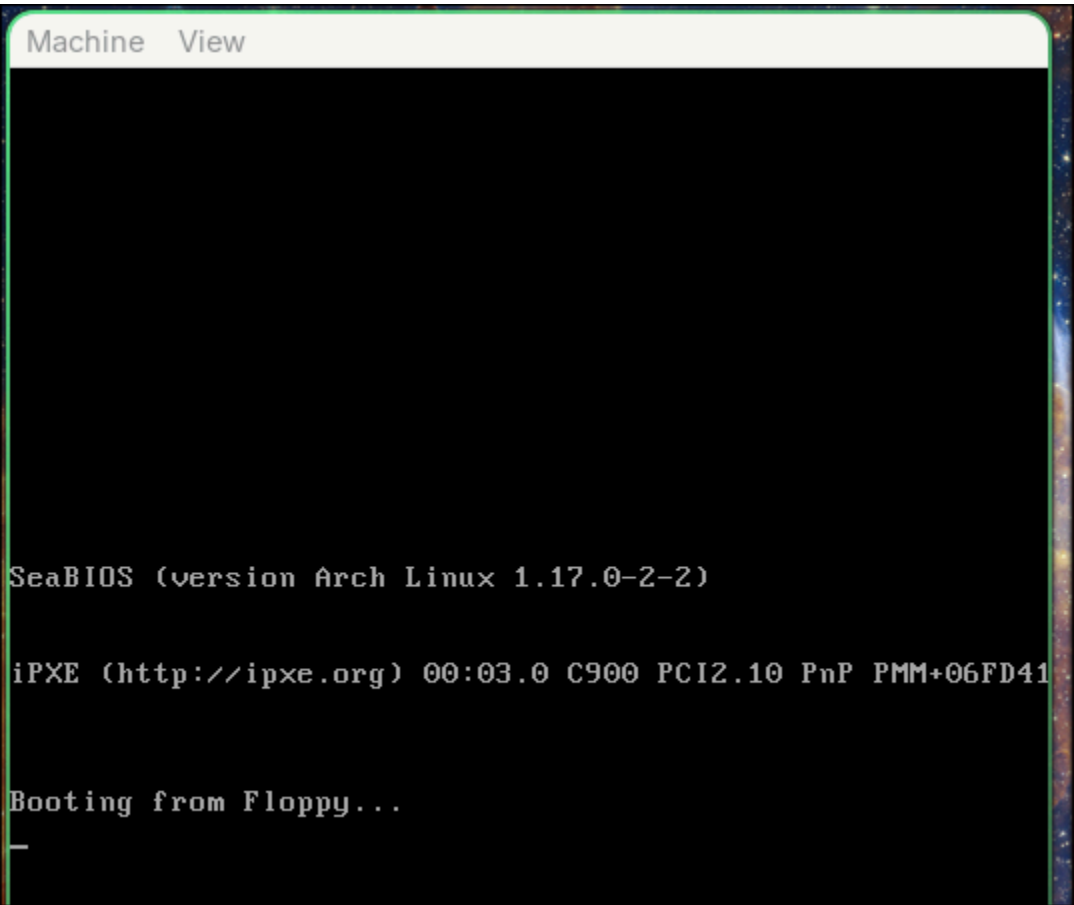
```
target remote localhost:1234
```

```
break *protected_mode
```

```
continue
```

Lo hacemos en 2 terminales porque en el terminal 1, se está emulando el hardware, y en el terminal 2 se está ejecutando el GDB, donde funciona como panel de control para inspeccionar la memoria y los registros. Si no lo abriéramos en 2 terminales, solo veríamos reiniciarse sin parar. Pero al usar el GDB aparte, podemos ver los registros CR0 y DS antes de que el sistema colapse.

Entonces, en un terminal, tenemos



```
Machine View

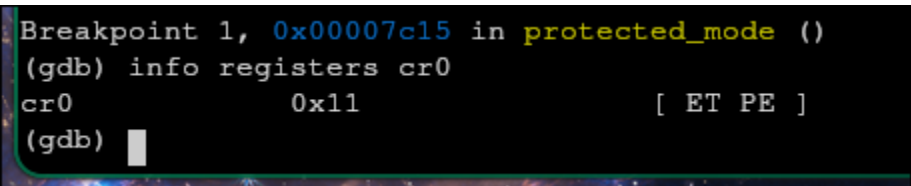
SeaBIOS (version Arch Linux 1.17.0-2-2)

iPXE (http://ipxe.org) 00:03.0 C900 PCI2.10 PnP PMM+06FD41

Booting from Floppy...
_
```

Y luego, con el otro terminal del GDB. Podemos ver los valores de los registros.

Escribiendo `info registers cr0`. El valor debe terminar en 1, en este caso nos muestra 0x11, eso prueba que el bit PE, protection enable, está activo. Lo que nos dice que el procesador paso de modo real y está en modo protegido validando cada acceso a memoria.



```
Breakpoint 1, 0x00007c15 in protected_mode ()
(gdb) info registers cr0
cr0                0x11                [ ET PE ]
(gdb) 
```

Escribiendo `info registers ds`. El valor tiene que ser 0x10. Eso prueba que se cargo selector de datos de tu GDT. En el cual solo configuramos específicamente con atributos de solo lectura.

```
(gdb) info registers ds
ds                0x10                16
(gdb) █
```

Este test completo, en la primera parte vemos que paso a modo protegido, donde al escribir un segmento que se configuró manualmente como solo lectura, el procesador se da cuenta de la instrucción y genera un fallo de protección. Lo cual hace que se reinicie constantemente el QEMU, y con la segunda parte, confirmamos exactamente con los registros que está en modo protegido, vemos también que el segmento está en solo lectura. Todo esto antes de que se reinicie por las instrucciones que le cargamos al GDB.

Conclusiones.

El desarrollo de esta actividad permitió validar la transición de la arquitectura x86 desde el modo real hacia el modo protegido, Aprendiendo como el hardware toma el control de la seguridad del sistema mediante la segmentación.

Aprendimos a utilizar herramientas como QEMU y GDB, donde pudimos comprender que el reinicio constante no era un problema de nuestra configuración o que el sistema no funcionaba, sino que nos representaba realmente lo que estaba pasando, el hardware bloqueando operaciones que no se podían hacer en segmentos configurados como solo lectura.

Por otro lado, el desarrollo del programa del desafío final lo desarrollamos en base a los ejemplos del repositorio x86-bare-metal-examples, donde vimos la estructura del programa, interrupciones, implementación de la GDT, activación del modo protegido y una serie de pasos que se necesitan para que el programa funcione. Esto nos permite ver la interacción real entre el programa y los registros del procesador.